
Guía de referencia · Data Science & ML

Machine Learning

Regresión · Clasificación · SVM · KNN · Clustering
PCA · Naïve Bayes · Random Forest · Modelos Generativos

v1.0 · 2025

Índice

1. Regresión

- 1.1. ¿Qué es la Regresión?
- 1.2. Regresión Lineal Simple
 - 1.2.1. Parámetros del modelo
 - 1.2.2. Entrenamiento: función de coste
 - 1.2.3. Solución analítica (Mínimos Cuadrados)
 - 1.2.4. Implementación con scikit-learn
- 1.3. Regresión Lineal Multivariable
- 1.4. Métricas de Evaluación
- 1.5. Regresión No Lineal
 - 1.5.1. Transformaciones simples
 - 1.5.2. Expansión polinómica
 - 1.5.3. Métodos Kernel
 - 1.5.4. Support Vector Machines (SVM)
 - 1.5.5. Procesos Gaussianos (GP)

2. Clasificación (I)

- 2.1. ¿Qué es la Clasificación?
- 2.2. Tipos de Clasificación
- 2.3. Encoding de Categorías (variable objetivo)
- 2.4. Métricas de Evaluación
 - 2.4.1. Conceptos base: TP, TN, FP, FN
 - 2.4.2. Métricas derivadas
 - 2.4.3. Curva ROC y AUC
 - 2.4.4. Binary Cross Entropy (BCE)
- 2.5. Técnicas de Clasificación
- 2.6. Regresión Logística
 - 2.6.1. Función sigmoide
 - 2.6.2. Entrenamiento: Descenso por Gradiente (SGD)
 - 2.6.3. Implementación con scikit-learn
- 2.7. Naïve Bayes
 - 2.7.1. Teorema de Bayes
 - 2.7.2. Implementación con scikit-learn

3. Clasificación (II)

- 3.1. Separación No Lineal
- 3.2. Transformaciones Simples para Clasificación
- 3.3. Expansión Polinómica para Clasificación
- 3.4. Support Vector Machines (SVM)
 - 3.4.1. Margen máximo y optimización
 - 3.4.2. SVM con datos no separables (soft margin)
 - 3.4.3. SVM con kernels
 - 3.4.4. Implementación con scikit-learn

- 3.5. K-Nearest Neighbors (KNN)
- 3.6. Gaussian Process Classifier (GPC)
- 3.7. Árboles de Decisión
 - 3.7.1. Índice Gini vs Entropía
 - 3.7.2. Implementación con scikit-learn
- 3.8. Random Forest
 - 3.8.1. ¿Qué es un ensemble?
 - 3.8.2. Implementación con scikit-learn
- 3.9. Clasificación Multiclase: OvR y OvO

4. Clustering

- 4.1. Aprendizaje No Supervisado
- 4.2. Clustering: Definición y Casos de Uso
- 4.3. Técnicas de Clustering
- 4.4. K-Means
 - 4.4.1. Algoritmo
 - 4.4.2. Selección de K: Método del Codo
 - 4.4.3. Implementación con scikit-learn
- 4.5. Gaussian Mixture Models (GMM)
 - 4.5.1. Algoritmo EM
 - 4.5.2. Implementación con scikit-learn
- 4.6. DBSCAN
 - 4.6.1. Algoritmo y parámetros
 - 4.6.2. Implementación con scikit-learn
- 4.7. HDBSCAN
- 4.8. Mean Shift
 - 4.8.1. Implementación con scikit-learn
- 4.9. Comparativa de Algoritmos de Clustering

5. Reducción de Dimensionalidad y Modelos Generativos

- 5.1. Reducción de Dimensionalidad: Introducción y Casos de Uso
- 5.2. Principal Component Analysis (PCA)
 - 5.2.1. Intuición: el eje óptimo de proyección
 - 5.2.2. Componentes principales y varianza
 - 5.2.3. Implementación con scikit-learn
 - 5.2.4. Selección del número de componentes: varianza explicada
- 5.3. Modelos Generativos
 - 5.3.1. PCA Probabilístico
 - 5.3.2. Modelos generativos modernos

1. Regresión

1.1. ¿Qué es la Regresión?

La regresión es una técnica supervisada que modela la relación entre una o varias variables independientes (features) y una variable dependiente continua (target).

El objetivo es aprender una función f tal que:

$$\hat{y} = f(x_1, x_2, \dots, x_D)$$

donde $\hat{y}$ es el valor predicho. A diferencia de la clasificación, la salida es un número real, no una categoría.

Ejemplos típicos: precio de una vivienda, demanda eléctrica, temperatura futura.

1.2. Regresión Lineal Simple

La forma más básica asume que la relación entre la variable de entrada x y la salida y es lineal:

$$\hat{y} = b + w * x$$

Parámetros del modelo

- b (bias / término independiente): desplazamiento vertical de la recta (punto de corte con el eje y). No multiplica a ninguna variable.
- w (weight / pendiente): tasa de variación de la salida respecto a la entrada. Indica cuánto cambia $\hat{y}$ por cada unidad de x .

Entrenamiento: función de coste

El aprendizaje automático consiste en encontrar los valores de b y w que minimizan el error sobre los datos de entrenamiento.

El error individual para una observación es el error cuadrático:

$$L = (y - \hat{y})^2$$

Se eleva al cuadrado para que siempre sea positivo. Sumando sobre todos los N ejemplos:

$$L(b, w) = \sum_{n=1}^N (y_n - \hat{y}_n)^2$$

El objetivo es encontrar los parámetros óptimos b^* y w^* que minimizan esta función de coste:

$$b^*, w^* = \{b, w\}_{\text{argmin}} L(b, w)$$

La solución se obtiene derivando e igualando a cero:

$$\frac{dL}{dw} = 0$$

Solución analítica (Mínimos Cuadrados)

La solución cerrada de Mínimos Cuadrados (Least Squares) es:

$$w^* = (X^T X)^{-1} X^T Y$$

donde X es la matriz de features (N x D, con una columna de unos para absorber b) e Y el vector de targets (N x 1). En la práctica no hay que implementar esto a mano: scikit-learn lo gestiona internamente.

Implementación con scikit-learn

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

modelo = LinearRegression()
modelo.fit(X_train, y_train)

y_pred = modelo.predict(X_test)

print(f"Bias (b): {modelo.intercept_: .4f}")
print(f"Pesos (w): {modelo.coef_}")
```

1.3. Regresión Lineal Multivariable

Cuando hay D variables de entrada, el modelo se extiende a:

$$y_{\text{hat}} = b + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

Cada peso w_i indica cuánto cambia la salida por cada unidad de la variable x_i , manteniendo el resto constantes.

Ejemplo — precio de vivienda

Variable	Peso	Interpretación
$x_1 = m^2$	$w_1 = 3$	+3.000 € por cada m^2 adicional
$x_2 = \text{distancia al centro (km)}$	$w_2 = -10$	-10.000 € por cada km de distancia

```
from sklearn.linear_model import LinearRegression
import pandas as pd

modelo = LinearRegression()
modelo.fit(X_train, y_train)

coefs = pd.Series(modelo.coef_, index=X_train.columns)
print(coefs.sort_values())
```

Con más de 2 variables ya no es posible visualizar la regresión directamente. Se usan gráficos de residuos o representaciones parciales (una variable vs. predicción, con el resto fijo).

1.4. Métricas de Evaluación

Lo fundamental no es solo conocer las métricas, sino saber cuándo usar cada una.

Error Absoluto Medio (MAE)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Robusto ante valores atípicos. Recomendado cuando el dataset contiene anomalías.

Error Medio Cuadrático (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Penaliza errores grandes más que el MAE. Es diferenciable, lo que lo hace útil durante el entrenamiento.

Raíz del Error Medio Cuadrático (RMSE)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Igual que MSE pero mantiene las unidades originales del problema. El más interpretable en la práctica.

Porcentaje de Error Absoluto Medio (MAPE)

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left(\frac{|y_i - \hat{y}_i|}{y_i} \right)$$

Expresa el error en porcentaje. Útil para comparar modelos sobre variables con distintas escalas.

Coefficiente de Determinación (R²)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Indica qué porcentaje de la variabilidad de los datos explica el modelo. R² = 1 es ajuste perfecto; R² = 0 equivale a predecir siempre la media. Especialmente útil para comparar modelos que predicen la misma variable.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"MAE: {mae:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R²: {r2:.4f}")
```

Guía de uso rápido

Métrica	Cuándo usarla
RMSE	Caso general; penaliza errores grandes; mantiene las unidades
MAE	Dataset con muchos outliers; interpretación directa
MAPE	Comparación entre variables de distinta escala
R ²	Comparar distintos modelos sobre la misma variable objetivo

1.5. Regresión No Lineal

Muchas relaciones reales no son lineales. El mapa de técnicas disponibles:

```
Regresión no lineal
├─ Transformaciones simples      (log, exp, sin...)
├─ Expansión polinómica
├─ Métodos Kernel
│   └─ SVM (Support Vector Machines)
│   └─ Procesos Gaussianos (GP)
└─ Modelos basados en árboles
    ├── Árboles de regresión
    ├── Random Forest
    └─ Gradient Boosting Machines
```

Los modelos basados en árboles (Random Forest, GBM) se cubren en detalle en el módulo de *Análítica Avanzada*.

Transformaciones simples

Cuando se conoce (o intuye) la forma funcional que siguen los datos, se puede transformar la variable antes de aplicar regresión lineal:

$$g(x) = \log(x) \quad g(x) = \exp(x) \quad g(x) = \sin(x)$$

El flujo es: $x \rightarrow g(x) \rightarrow \hat{y}$

El modelo resultante sigue siendo lineal en los parámetros b y w :

$$\hat{y} = b + w * g(x)$$

```
import numpy as np
from sklearn.linear_model import LinearRegression

X_log = np.log(X)
modelo = LinearRegression()
modelo.fit(X_log, y)
```

Expansión polinómica

Se amplía la matriz de datos con potencias de las variables originales, permitiendo capturar curvaturas sin salir del marco lineal:

$$\hat{y} = b + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + w_5 x^5$$

El grado del polinomio es un hiperparámetro a validar. A mayor grado, mayor riesgo de sobreajuste.

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('lr', LinearRegression())
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

```

Siempre usar `Pipeline` para evitar data leakage al transformar train y test por separado.

Métodos Kernel

Un kernel es una función que mide la similitud entre dos observaciones x y x' :

$$k(x, x')$$

Tipos de kernel habituales

Kernel	Fórmula	Uso
Squared Exponential (SE / RBF)	$\sigma_f^2 \exp(-(x-x')^2/(2l^2))$	Relaciones suaves y continuas
Periódico (Per)	$\sigma_f^2 \exp(-(2)/(l^2) \sin^2(\pi(x-x')/(p)))$	Datos con estacionalidad
Lineal (Lin)	$\sigma_f^2 (x - c)(x' - c)$	Equivalente a regresión lineal

Support Vector Machines (SVM)

SVM busca el hiperplano que mejor modela la tendencia de los datos, maximizando el margen entre las bandas paralelas que envuelven la nube de puntos.

Función objetivo

$$\max (1)/(2) |\omega| ^2 + C \sum_{(i=1)} ^{(N)} (x_i \cdot i + x_i \cdot i^*)$$

- C : equilibra la regularidad del modelo frente al error cometido (hiperparámetro clave).
- x_i : variables de holgura que controlan el error al aproximar los vectores de soporte.

```

from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf', C=1.0, epsilon=0.1))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

```

Ventajas y desventajas

Ventajas	Desventajas
Eficaz en espacios de alta dimensionalidad	Poco eficiente con datasets grandes
Flexible (múltiples kernels disponibles)	Elección del kernel puede ser difícil
Robusto frente a outliers	

Procesos Gaussianos (GP)

Un Proceso Gaussiano es un modelo no paramétrico y bayesiano que predice distribuciones completas sobre funciones, cuantificando la incertidumbre de cada predicción.

$$f(x) \sim GP(0, k(x, x'))$$

La media y covarianza de las predicciones son:

$$E[f_*] = K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} y$$

$$\text{Cov}[f_*] = K(X_*, X_*) - K(X_*, X)(K(X, X) + \sigma^2 I)^{-1} K(X, X_*)$$

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel

kernel = RBF(length_scale=1.0) + WhiteKernel(noise_level=0.1)
gp = GaussianProcessRegressor(kernel=kernel, random_state=42)
gp.fit(X_train, y_train)

y_pred, y_std = gp.predict(X_test, return_std=True)
```

Ventajas y desventajas

Ventajas	Desventajas
Proporciona intervalos de confianza en la predicción	Escala mal con datasets grandes ($O(N^3)$)
Muy flexible y no paramétrico	El kernel es difícil de elegir
Ideal cuando los datos son escasos pero críticos	Los resultados son sensibles al ajuste del kernel

2. Clasificación (I)

2.1. ¿Qué es la Clasificación?

La clasificación es una técnica de aprendizaje supervisado que consiste en predecir una etiqueta (categoría) para unos datos de entrada. A diferencia de la regresión, la salida no es un número continuo sino una clase discreta.

Casos de uso habituales

Tipo	Ejemplos
Binaria (2 clases)	Detección de spam, fraude, diagnóstico médico
Multiclase (>2 clases)	Detección de objetos, análisis de sentimiento, segmentación

La clave para identificar un problema de clasificación es que la variable objetivo toma valores de un conjunto finito de categorías — no valores continuos.

2.2. Tipos de Clasificación

Clasificación binaria — la salida es una de dos clases (positivo/negativo, spam/no-spam, 0/1).

Clasificación multiclase — la salida es una de $K > 2$ clases mutuamente excluyentes. Cada observación pertenece exactamente a una clase.

Clasificación multi-etiqueta — cada observación puede pertenecer simultáneamente a varias clases. Las etiquetas se representan como un vector binario: `[1, 1, 0]` indica que pertenece a las clases 1 y 2 pero no a la 3.

```
Binaria:      [0] o [1]
Multiclase:   [1, 0, 0] o [0, 1, 0] o [0, 0, 1]
Multi-etiqueta: [1, 1, 0] o [0, 1, 1] o [1, 0, 1]
```

2.3. Encoding de Categorías (variable objetivo)

Antes de entrenar un modelo, las etiquetas de texto deben convertirse en valores numéricos. Hay tres estrategias principales:

One-hot encoding — crea una columna binaria por cada categoría. Recomendado para variables nominales (sin orden).

```
import pandas as pd

df = pd.get_dummies(df, columns=['nivel_ahorros'])
```

Label encoding — asigna un entero arbitrario a cada categoría. Útil para árboles de decisión, pero introduce orden artificial en algoritmos lineales.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['nivel_cod'] = le.fit_transform(df['nivel_ahorros'])
```

Ordinal encoding — asigna enteros respetando un orden explícito. Solo usar cuando la variable tiene orden natural.

```
from sklearn.preprocessing import OrdinalEncoder

oe = OrdinalEncoder(categories=[['Bajo', 'Medio', 'Alto']])
df[['nivel_cod']] = oe.fit_transform(df[['nivel_ahorros']])
```

Nunca usar Label Encoding con variables nominales en modelos lineales o de distancias (KNN, SVM): el modelo interpretará que "Bajo=2 > Medio=1" tiene significado matemático.

2.4. Métricas de Evaluación

Conceptos base: TP, TN, FP, FN

Todo clasificador binario produce cuatro tipos de resultados:

Símbolo	Nombre	Descripción
TP	True Positive	Real=Positivo, predicho=Positivo ☐
TN	True Negative	Real=Negativo, predicho=Negativo ☐
FP	False Positive	Real=Negativo, predicho=Positivo ☐
FN	False Negative	Real=Positivo, predicho=Negativo ☐

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot()
```

Métricas derivadas

Accuracy — proporción de predicciones correctas sobre el total:

$$Acc = (TP + TN) / (TP + TN + FP + FN)$$

Con clases desbalanceadas, la accuracy puede ser engañosa.

Precision — de todos los predichos como positivos, ¿cuántos lo son realmente?

$$Precision = (TP) / (TP + FP)$$

Recall (Exhaustividad) — de todos los positivos reales, ¿cuántos detecta el modelo?

$$Recall = (TP) / (TP + FN)$$

Especificidad — de todos los negativos reales, ¿cuántos identifica correctamente?

$$Especificidad = (TN) / (TN + FP)$$

F1-Score — media armónica de Precision y Recall:

$$F1 = 2 * (Recall * Precision) / (Recall + Precision)$$

Guía de cuándo usar cada métrica

Métrica	Cuándo priorizarla
Accuracy	Clases balanceadas, visión general
Precision	Coste alto de falsos positivos (ej. spam)
Recall	Coste alto de falsos negativos (ej. detección de cáncer)
F1	Desbalanceo de clases; cuando importan tanto FP como FN

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=['Neg', 'Pos']))
```

Curva ROC y AUC

La curva ROC representa la tasa de verdaderos positivos (Recall) frente a la tasa de falsos positivos (1 - Especificidad) para todos los umbrales posibles. El AUC resume la curva en un único número: AUC = 1.0 es clasificador perfecto; AUC = 0.5 equivale a clasificar aleatoriamente.

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

y_prob = modelo.predict_proba(X_test)[: , 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

Binary Cross Entropy (BCE)

La BCE es la función de coste utilizada para entrenar clasificadores binarios. Penaliza fuertemente la confianza en predicciones incorrectas.

$$BCE = -(y * \log f(x) + (1 - y) * \log(1 - f(x)))$$

Para todos los ejemplos del conjunto de entrenamiento:

$$L(b, w) = -(1)/(N) \sum_{n=1}^N (y_n * \log f(x_n) + (1 - y_n) * \log(1 - f(x_n)))$$

2.5. Técnicas de Clasificación

```

Clasificación
├─ Lineal
│   ├── Regresión Logística
│   └─ Naïve Bayes
└─ No lineal
    ├── Transformaciones simples / polinómicas
    ├── Métodos Kernel (SVM, Gaussian Processes)
    ├── Modelos basados en árboles
    │   ├── Árboles de decisión
    │   ├── Random Forest
    │   └─ Gradient Boosting Machines
    └─ Redes neuronales

```

2.6. Regresión Logística

A pesar del nombre, es un algoritmo de clasificación. Parte del mismo modelo lineal de la regresión pero añade una función que transforma la salida en una probabilidad entre 0 y 1.

Función sigmoide

Para obtener probabilidades, se aplica la función sigmoide sigma a la salida lineal:

$$f(x) = \text{sigma}(b + w * x) = (1)/(1 + \exp(-(b + w * x)))$$

- $f(x) \geq 0.5 \rightarrow$ clase positiva
- $f(x) < 0.5 \rightarrow$ clase negativa

Entrenamiento: Descenso por Gradiente (SGD)

La sigmoide elimina la posibilidad de solución cerrada. Se usa Descenso por Gradiente Estocástico (SGD):

$$b_{(t+1)}, w_{(t+1)} = b_t, w_t - \text{eta} \nabla_{(b,w)} L(b, w; x_i, y_i)$$

Learning rate	Efecto
Demasiado bajo	Convergencia lenta
Adecuado	Converge rápidamente al mínimo
Demasiado alto	Actualizaciones divergentes

Implementación con scikit-learn

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

modelo = LogisticRegression(max_iter=1000, random_state=42)
modelo.fit(X_train, y_train)

y_pred = modelo.predict(X_test)
y_proba = modelo.predict_proba(X_test)[: , 1]

print(classification_report(y_test, y_pred))
print(f"Pesos (w): {modelo.coef_}")
print(f"Bias (b): {modelo.intercept_}")

```

Ventajas y desventajas

Ventajas	Desventajas
Simple y rápido de entrenar	Modelo lineal: poca flexibilidad
Coefficientes interpretables	No captura relaciones entre variables
Produce probabilidades	Sensible a outliers

2.7. Naïve Bayes

Clasificador probabilístico basado en el teorema de Bayes. Asume que todas las features son independientes entre sí dado el valor de la clase.

Teorema de Bayes

$$P(y|D) = (P(D|y) * P(y)) / (P(D))$$

- $P(y|D)$ — posterior: probabilidad de la clase dado D.
- $P(D|y)$ — likelihood: probabilidad de observar D si la clase es y.
- $P(y) = \{n_y\}/\{N\}$ — prior: frecuencia de la clase en el entrenamiento.

La predicción elige la clase con mayor probabilidad posterior:

$$y_{\text{hat}} = \text{argmax}_y (P(D|y) * P(y)) / (P(D))$$

Elección del likelihood: features discretas → Multinomial NB; features continuas → Gaussian NB.

Implementación con scikit-learn

```

from sklearn.naive_bayes import GaussianNB

modelo = GaussianNB()
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)

```

Ejemplo con texto (clasificación de spam)

```

from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('clf', MultinomialNB())
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

```

Ventajas y desventajas

Ventajas	Desventajas
Muy rápido de entrenar	Asume independencia entre features (raramente cierto)
Funciona bien con datos escasos	Puede no ajustarse bien a los datos
Escala bien con alta dimensionalidad (ej. NLP)	Requiere elegir likelihood y prior

3. Clasificación (II)

3.1. Separación No Lineal

Muchos problemas reales no son linealmente separables: no existe un único hiperplano que divida perfectamente las clases. Esto puede ocurrir con una o varias variables de entrada.

Con una variable, la clase positiva puede ocupar dos regiones distintas (ej. valores bajos y altos de x , con la clase negativa en el centro), lo que un modelo lineal no puede capturar. Con dos variables, las clases pueden estar dispuestas en patrones circulares, espirales u otras formas complejas que requieren fronteras de decisión curvas.

La solución pasa por aplicar transformaciones a las features antes de entrenar el clasificador lineal.

3.2. Transformaciones Simples para Clasificación

Se aplica la misma idea que en regresión: transformar las variables de entrada con funciones conocidas (log, exp, sin, coordenadas polares, etc.) antes de pasar el resultado a la sigmoide.

Ejemplo 1 — dos regiones con x^2

$$f(x) = \text{sigma}(b + w_1 x + w_2 x^2) \text{ Accuracy: } 0.970 \text{ vs } 0.600 \text{ lineal}$$

Ejemplo 2 — datos periódicos con $\sin(x)$

$$f(x) = \text{sigma}(b + w_1 \sin(x)) \text{ Accuracy: } 0.850 \text{ vs } 0.660 \text{ lineal}$$

Ejemplo 3 — estructura circular con coordenadas polares

$$r(x_1, x_2) = \sqrt{x_1^2 + x_2^2} \quad \alpha(x_1, x_2) = \arctan((x_2)/(x_1))$$

$$f(x_1, x_2) = \text{sigma}(b + w_1 * r(x_1, x_2) + w_2 * \alpha(x_1, x_2)) \text{ Accuracy: } 0.890 \text{ vs } 0.507 \text{ lineal}$$

```
import numpy as np
from sklearn.linear_model import LogisticRegression

r = np.sqrt(X[:, 0]**2 + X[:, 1]**2).reshape(-1, 1)
alpha = np.arctan2(X[:, 1], X[:, 0]).reshape(-1, 1)
X_polar = np.hstack([r, alpha])

modelo = LogisticRegression()
modelo.fit(X_polar, y)
```

Las transformaciones simples requieren que el analista conozca o intuya la forma funcional de los datos.

3.3. Expansión Polinómica para Clasificación

Cuando no hay un patrón claro, la expansión polinómica genera sistemáticamente todas las combinaciones de potencias de las features hasta un grado dado.

Para dos variables x_1 y x_2 con grado 3:

$$f(x_1, x_2) = \text{sigma}(b + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 + w_6 x_1^3 + w_7 x_1^2 x_2 + w_8 x_1 x_2^2 + w_9 x_2^3)$$

Accuracy: 0.927 vs 0.863 lineal

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('poly', PolynomialFeatures(degree=3, include_bias=False)),
    ('clf', LogisticRegression(max_iter=1000))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

A mayor grado, mayor riesgo de sobreajuste. Validar el grado con cross-validation.

3.4. Support Vector Machines (SVM)

SVM busca el hiperplano que maximiza el margen de separación entre las clases.

Margen máximo y optimización

El problema de optimización se formula como:

$$\min_{(w, b)} \frac{1}{2} \|w\|^2 \text{ sujeto a } y_i (w^T x_i + b) \geq 1 \text{ for all } i$$

Forma dual (Lagrangiano):

$$\max_{(\alpha)} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \text{ s.a. } \sum_i \alpha_i y_i = 0, \alpha_i \geq 0$$

SVM con datos no separables (soft margin)

Se introduce la variable de holgura $\xi_i \geq 0$:

$$\min_w (w, b, \xi) \frac{1}{2} \|w\|^2 \text{ sujeto a } y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

El hiperparámetro C controla el equilibrio entre maximizar el margen y permitir errores.

SVM con kernels

Kernel	Fórmula		
Lineal	$K(x, x') = x \cdot x'$		
Polinómico	$K(x, x') = (x \cdot x' + 1)^d$		
RBF (Gaussiano)	$K(x, x') = \exp(-\gamma \ x - x'\ ^2)$	γ	γ
Sigmoide	$K(x, x') = \frac{1}{1 + \exp(-\alpha x \cdot x' + c)}$		

Implementación con scikit-learn

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', C=1.0, probability=True))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
y_proba = pipeline.predict_proba(X_test)[:, 1]
```

Ventajas y desventajas

Ventajas	Desventajas
Eficaz en espacios de alta dimensionalidad	Poco eficiente con datasets grandes
Flexible gracias a la variedad de kernels	Elección del kernel puede ser difícil
Robusto frente a outliers	

3.5. K-Nearest Neighbors (KNN)

KNN es un algoritmo no paramétrico y de lazy learning. Para clasificar un nuevo punto, busca sus K vecinos más cercanos y asigna la clase mayoritaria.

Algoritmo paso a paso:

1. Calcular la distancia euclídea a cada punto del dataset: $\text{dist}(x_{\text{new}}, x_i) = \|x_{\text{new}} - x_i\|_2$
2. Seleccionar los K puntos con menor distancia.
3. Asignar la clase más frecuente: $y_{\text{hat}}(\text{new}) = \text{moda}(\{y_i \mid i \text{ in } K \text{ vecinos ms cercanos}\})$

K es el hiperparámetro clave: un K pequeño produce sobreajuste; un K grande suaviza la frontera.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

Ventajas y desventajas

Ventajas	Desventajas
Muy fácil de entender e implementar	Lento en predicción con datasets grandes
No requiere entrenamiento (lazy learning)	Muy sensible a la escala de las variables
Funciona bien en problemas no lineales	Rendimiento degradado con alta dimensionalidad

3.6. Gaussian Process Classifier (GPC)

El GPC extiende el GP de regresión al problema de clasificación mediante una función latente $f(x)$ que sigue una distribución GP:

$$f(x) \sim GP(0, k(x, x'))$$

Esta función latente se transforma en probabilidad de clase 1 mediante la sigmoide:

$$p_i(x) = \text{sigma}(f(x)) = (1)/(1 + e^{-(f(x))})$$

La predicción requiere integrar sobre la distribución posterior de f_* :

$$p(y_* = 1 | x_*, D) = \int \text{sigma}(f_*) p(f_* | x_*, D) df_*$$

```
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF

kernel = 1.0 * RBF(length_scale=1.0)
gpc = GaussianProcessClassifier(kernel=kernel, random_state=0)
gpc.fit(X_train, y_train)
y_pred = gpc.predict(X_test)
y_proba = gpc.predict_proba(X_test)
```

Las ventajas y desventajas del GPC son las mismas que las del GP de regresión: proporciona incertidumbre en la predicción, es muy flexible, pero escala mal con N grande y su rendimiento depende fuertemente del kernel elegido.

3.7. Árboles de Decisión

Un árbol de decisión divide recursivamente el espacio de features mediante preguntas binarias. En cada división, el algoritmo elige el corte que maximiza la pureza de los nodos hijos.

Índice Gini vs Entropía

Índice Gini — mide la impureza de un nodo:

$$Gini(t) = 1 - \sum_{i=1}^n p_i^2$$

Entropía — mide el desorden del sistema:

$$Entropia = -\sum_{i=1}^n p_i \cdot \log_2 p_i$$

Ambas producen resultados similares. Gini es más rápido de calcular; Entropía tiende a generar árboles más balanceados.

Implementación con scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import classification_report
import pandas as pd

modelo = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)

print(classification_report(y_test, y_pred))
print(export_text(modelo, feature_names=list(X_train.columns)))

importancias = pd.Series(modelo.feature_importances_, index=X_train.columns)
print(importancias.sort_values(ascending=False))
```

Ventajas y desventajas

Ventajas	Desventajas
Fácil de interpretar y visualizar	Propenso al sobreajuste sin poda
Requiere poca preparación de datos	Alta varianza: pequeños cambios → árbol muy diferente

3.8. Random Forest

Random Forest es un método de ensemble que combina múltiples árboles de decisión entrenados sobre subconjuntos aleatorios de datos y features. La predicción final es la clase mayoritaria (majority voting).

¿Qué es un ensemble?

Bagging (Random Forest) — cada modelo base se entrena sobre un subconjunto aleatorio del dataset (con reemplazo). Reduce la varianza.

Boosting — cada modelo base se entrena corrigiendo los errores del anterior. Reduce el sesgo.

Stacking — se entrenan modelos base heterogéneos y un meta-modelo aprende a combinar sus predicciones.

Implementación con scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import pandas as pd

modelo = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    max_depth=None,
    n_jobs=-1,
    class_weight='balanced',
    random_state=42
)
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
print(classification_report(y_test, y_pred))

importancias = pd.Series(
    modelo.feature_importances_, index=X_train.columns
).sort_values(ascending=False)
print(importancias)
```

Parámetros clave

Parámetro	Descripción
<code>n_estimators</code>	Número de árboles del bosque
<code>criterion</code>	Métrica de pureza: <code>'gini'</code> o <code>'entropy'</code>
<code>max_depth</code>	Profundidad máxima de cada árbol
<code>n_jobs</code>	Procesadores en paralelo (<code>-1</code> = todos)
<code>class_weight</code>	Pesos por clase para datasets desbalanceados

Ventajas y desventajas

Ventajas	Desventajas
Gran rendimiento y baja varianza	Alto coste computacional
Alta estabilidad y robustez	Alta correlación entre árboles con datasets pequeños
Proporciona importancia de variables	

3.9. Clasificación Multiclase: OvR y OvO

Muchos clasificadores binarios pueden extenderse a $K > 2$ clases mediante dos estrategias:

One-vs-Rest (OvR) — se entrena un clasificador binario por cada clase: "clase k" vs "todas las demás". Para K clases se entrenan K modelos.

One-vs-One (OvO) — se entrena un clasificador por cada par de clases. Para K clases se entrenan $\{K(K-1)\}/2$ modelos.

Nativos — KNN, árboles de decisión y redes neuronales admiten multiclase de forma nativa.

```
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

base_clf = LogisticRegression(max_iter=1000)

clf_ovr = OneVsRestClassifier(base_clf)
clf_ovr.fit(X_train, y_train)
print(f"OvR Accuracy: {accuracy_score(y_test, clf_ovr.predict(X_test)):.3f}")

clf_ovo = OneVsOneClassifier(base_clf)
clf_ovo.fit(X_train, y_train)
print(f"OvO Accuracy: {accuracy_score(y_test, clf_ovo.predict(X_test)):.3f}")
```

En scikit-learn, `LogisticRegression` y `SVC` ya aplican OvR por defecto cuando `y` tiene más de dos clases.

4. Clustering

4.1. Aprendizaje No Supervisado

En el aprendizaje no supervisado, el modelo recibe datos de entrada x pero no dispone de etiquetas y . El objetivo de aprendizaje depende del tipo de aplicación:

En clustering, se minimiza la distancia de cada muestra a su cluster asignado:

$$L = (1)/(N) \sum_{k=1}^K \sum_{i: A(x^{(i)})=k} d(x^{(i)}, c_k)$$

En reducción de dimensionalidad, se minimiza el error de reconstrucción:

$$J = (1)/(N) \sum_{n=1}^N D \|x_n - \{x\}_n\|_2^2$$

En modelos generativos, se maximiza la verosimilitud de que el modelo haya generado los datos observados:

$$L = -\sum_{n=1}^N \log p_{\theta}(x)$$

4.2. Clustering: Definición y Casos de Uso

El clustering (o agrupamiento) agrupa datos similares en conjuntos llamados clusters. La similitud se define en función de las variables de entrada.

Casos de uso habituales

Dominio	Aplicación
Marketing / banca	Segmentación de clientes
Retail / e-commerce	Estrategias de fidelización por segmento
Medios digitales	Agrupación automática de noticias
Atención al cliente	Clasificación de tickets de soporte
Ciberseguridad	Detección de accesos anómalos
Sanidad	Monitorización remota de pacientes
Geolocalización	Zonas de alta demanda de taxis

4.3. Técnicas de Clustering

- K-Means — basado en centroides, asume clusters esféricos de tamaño similar.
- Gaussian Mixture Models (GMM) — basado en distribuciones, captura formas elípticas y asigna probabilidades de pertenencia.
- DBSCAN — basado en densidad, detecta clusters de forma arbitraria y etiqueta outliers explícitamente.
- HDBSCAN — extensión jerárquica de DBSCAN, más robusta y eficiente.
- Mean Shift — basado en densidad, no requiere especificar K ni epsilon.

4.4. K-Means

K-Means es el algoritmo de clustering más sencillo y ampliamente utilizado. Su objetivo es encontrar K centroides minimizando:

$$L = \frac{1}{N} \sum_{k=1}^K \sum_{i: A(x^{(i)})=k} d(x^{(i)}, c_k)$$

Algoritmo

Inicialización: se colocan K centroides aleatoriamente (o con K-Means++ para una inicialización más inteligente).

Paso 1 — Asignación: para cada punto, se encuentra el centroide más cercano.

Paso 2 — Actualización: se recalcula cada centroide como la media de todos los puntos asignados.

Se repite hasta convergencia (centroides estables).

Selección de K: Método del Codo

Se evalúa la función de coste $L(K)$ para distintos valores de K y se busca el "codo de la curva": el punto a partir del cual añadir más clusters apenas reduce el coste.

```

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

costes = []
rango_k = range(2, 15)

for k in rango_k:
    km = KMeans(n_clusters=k, random_state=42, n_init='auto')
    km.fit(X)
    costes.append(km.inertia_)

plt.plot(rango_k, costes, 'o-')
plt.xlabel('K')
plt.ylabel('Coste L(K)')
plt.title('Método del Codo')
plt.show()

```

El codo no siempre es obvio. Complementar con el coeficiente de silueta.

Implementación con scikit-learn

```

from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=6, random_state=42, n_init='auto')
kmeans.fit(X)

y_kmeans = kmeans.predict(X)
centros = kmeans.cluster_centers_

```

Ventajas y desventajas

Ventajas	Desventajas
Fácil de implementar y muy eficiente	Requiere especificar K de antemano
Converge en pocas iteraciones	Sensible a la inicialización aleatoria
Centroide interpretable	No funciona con clusters de formas arbitrarias
Escala bien a muchas dimensiones	Sensible a outliers

4.5. Gaussian Mixture Models (GMM)

Los GMM asumen que los datos provienen de una mezcla de distribuciones gaussianas:

$$p(x) = \sum_{k=1}^K \pi_k N(x | \mu_k, \Sigma_k)$$

A diferencia de K-Means, los GMM asignan una probabilidad de pertenencia a cada componente (asignación blanda).

Algoritmo EM

Los GMM se entrenan con el algoritmo Expectation-Maximization (EM):

E-step: estimar las probabilidades de pertenencia de cada punto a cada componente.

M-step: actualizar los parámetros μ_k , Σ_k y π_k maximizando la verosimilitud ponderada.

Implementación con scikit-learn

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
gmm.fit(X)

y_gmm = gmm.predict(X)
probs = gmm.predict_proba(X)  # shape (N, K)
```

Opciones de `covariance_type` : 'full' , 'tied' , 'diag' , 'spherical' .

Ventajas y desventajas

Ventajas	Desventajas
Captura formas elípticas	Requiere especificar K
Asignación probabilística	Sensible a la inicialización
Útil en inferencia bayesiana	No funciona bien con clusters de formas muy arbitrarias

4.6. DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) no requiere especificar K y etiqueta explícitamente los outliers.

Algoritmo y parámetros

Opera con dos hiperparámetros: epsilon (radio de vecindad) y `min_samples` (mínimo de vecinos para punto central).

1. Se cuenta cuántos puntos existen a distancia $\leq \epsilon$.
2. Si hay al menos `min_samples` vecinos, el punto es un punto central.
3. Los puntos sin vecino central cercano se etiquetan como anomalías (label = -1).

La distancia euclídea utilizada: $d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$

Elección de epsilon: fijar de forma que al menos el 90 % de las observaciones tengan un vecino a distancia $d \leq \epsilon$.

Implementación con scikit-learn


```

from sklearn.cluster import DBSCAN
import numpy as np

dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan.fit(X)

y_dbscan = dbscan.labels_
n_clusters = len(set(y_dbscan)) - (1 if -1 in y_dbscan else 0)
n_outliers = np.sum(y_dbscan == -1)
print(f"Clusters: {n_clusters} | Outliers: {n_outliers}")

```

Para elegir `eps` automáticamente: graficar la distancia al k-ésimo vecino y buscar el codo.

```

from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt

nn = NearestNeighbors(n_neighbors=5)
nn.fit(X)
distancias, _ = nn.kneighbors(X)
distancias_ordenadas = np.sort(distancias[:, -1])

plt.plot(distancias_ordenadas)
plt.xlabel('Puntos ordenados')
plt.ylabel('Distancia al 5º vecino')
plt.title('Elección de eps')
plt.show()

```

Ventajas y desventajas

Ventajas	Desventajas
No requiere especificar K	Difícil de escalar a alta dimensión
Detecta clusters de forma arbitraria	Sensible a <code>eps</code> y <code>min_samples</code>
Detecta y etiqueta outliers explícitamente	No funciona bien con densidades muy diferentes

4.7. HDBSCAN

HDBSCAN (Hierarchical DBSCAN) es una generalización jerárquica de DBSCAN. En lugar de usar un epsilon fijo, ejecuta DBSCAN para un rango de valores de epsilon y extrae la solución más estable, siendo más robusto cuando los clusters tienen densidades variables.

```

import hdbscan # pip install hdbscan

clusterer = hdbscan.HDBSCAN(min_cluster_size=10)
clusterer.fit(X)

y_hdbscan = clusterer.labels_
scores = clusterer.outlier_scores_

```

En versiones recientes de scikit-learn también está disponible como `sklearn.cluster.HDBSCAN`.

4.8. Mean Shift

Mean Shift localiza los máximos de densidad del espacio de features sin necesidad de especificar K ni epsilon.

1. Estimación de densidad: se estima la función de densidad de probabilidad.
2. Desplazamiento iterativo: cada punto se mueve hacia la media de los puntos dentro de una ventana de radio `bandwidth`.
3. Convergencia: la posición final del punto representa el centroide del cluster.

Implementación con scikit-learn

```
from sklearn.cluster import MeanShift, estimate_bandwidth
import numpy as np

bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=500)

ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(X)

y_ms = ms.labels_
centros = ms.cluster_centers_
print(f"Clusters encontrados: {len(np.unique(y_ms))}")
```

Ventajas y desventajas

Ventajas	Desventajas
No requiere especificar K	Alta complejidad computacional
Detecta clusters de forma arbitraria	Muy sensible al parámetro <code>bandwidth</code>
Robusto a la inicialización	No detecta outliers explícitamente

4.9. Comparativa de Algoritmos de Clustering

Algoritmo	¿Requiere K?	Forma de clusters	Outliers	Escalabilidad
K-Means	Sí	Esférica / convexa	No detecta	Alta
GMM	Sí	Elíptica arbitraria	No detecta	Media
DBSCAN	No	Arbitraria	Detecta y etiqueta	Media-baja
HDBSCAN	No	Arbitraria	Detecta y puntúa	Media
Mean Shift	No	Arbitraria	No detecta	Baja

Guía rápida de elección

- Clusters bien separados y aproximadamente esféricos → K-Means.
- Clusters con formas elípticas o solapados → GMM.
- Forma desconocida, presencia de outliers, K no conocido → DBSCAN o HDBSCAN.
- Sin parámetros que ajustar y dataset no demasiado grande → Mean Shift.

5. Reducción de Dimensionalidad y Modelos Generativos

5.1. Reducción de Dimensionalidad: Introducción y Casos de Uso

La reducción de dimensionalidad transforma los datos de entrada a un espacio de menor dimensión, minimizando el error de reconstrucción:

$$J = \frac{1}{N} \sum_{n=1}^N \|x_n - \hat{x}_n\|_2^2$$

Motivaciones principales

Motivación	Descripción	Casos reales
Compresión de datos	Almacenar y transmitir con menos bits	Imágenes (WhatsApp, Instagram), audio MP3, IoT
Reducción de ruido	Eliminar componentes irrelevantes	Reconocimiento facial, señales ECG/EEG
Reducción del sobreajuste	Quedarse con las features más informativas	Diagnóstico médico, NLP con PCA/LSA
Eficiencia computacional	Modelos más ligeros y rápidos	ML en móviles, sensores IoT, datasets masivos
Visualización	Proyectar a 2D/3D para exploración	Exploración de datos complejos, detección de outliers

5.2. Principal Component Analysis (PCA)

PCA proyecta linealmente los datos a un espacio de menor dimensión buscando las direcciones que maximizan la varianza (equivalente a minimizar el error de reconstrucción). Cada dirección óptima se denomina componente principal.

Intuición: el eje óptimo de proyección

Dado un dataset 2D que queremos reducir a 1D, buscamos el eje de proyección que minimice la información perdida. El eje óptimo es el que se alinea con la dirección de mayor dispersión de los puntos, minimizando las líneas de error entre cada punto y su proyección.

La clave: menor error de reconstrucción $\leftrightarrow$ mayor varianza en el espacio transformado. PCA encuentra ese eje óptimo automáticamente mediante álgebra lineal.

Componentes principales y varianza

PCA resuelve el problema de autovectores de la matriz de covarianza. La transformación completa se representa como una matriz W in $\mathbb{R}^{(D \times d)}$:

$$W = [w_{(11)} \ w_{(12)} \ \dots \ w_{(1d)} \ w_{(21)} \ w_{(22)} \ \dots \ w_{(2d)} \ \dots \ w_{(D1)} \ w_{(D2)} \ \dots \ w_{(Dd)}]$$

Las columnas de W son los autovectores de la matriz de covarianza, ordenados de mayor a menor varianza explicada.

Implementación con scikit-learn

```
from sklearn.decomposition import PCA
import numpy as np
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
pca.fit(X)

print(pca.components_)      # shape (n_components, n_features)
print(pca.explained_variance_)

Z = pca.transform(X)        # proyección al espacio reducido
X_rec = pca.inverse_transform(Z) # reconstrucción
```

Para visualizar cualquier dataset en 2D:

```
pca_2d = PCA(n_components=2)
Z_2d = pca_2d.fit_transform(X)

plt.scatter(Z_2d[:, 0], Z_2d[:, 1], c=y, cmap='tab10')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Preprocesado obligatorio: PCA es sensible a la escala. Siempre aplicar `StandardScaler` antes.

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95)) # conservar el 95% de la varianza
])
Z = pipe.fit_transform(X)
```

Selección del número de componentes: varianza explicada

La varianza explicada acumulada indica qué fracción de la información total retiene el modelo con K componentes:

$$F(K) = \frac{\sum_{k=1}^K \lambda_k}{\sum_{k=1}^D \lambda_k}$$

```
import numpy as np
import matplotlib.pyplot as plt

pca_full = PCA()
pca_full.fit(X)

varianza_acumulada = np.cumsum(pca_full.explained_variance_) / np.sum(pca_full.explained_variance_)

plt.plot(range(1, len(varianza_acumulada) + 1), varianza_acumulada, 'o-')
plt.axhline(0.90, color='red', linestyle='--', label='90% varianza')
plt.xlabel('Número K de componentes principales')
plt.ylabel('F(K) - Varianza acumulada')
plt.legend()
plt.show()

k_90 = np.argmax(varianza_acumulada >= 0.90) + 1
print(f"Componentes necesarias para el 90% de varianza: {k_90}")
```

Guía de elección del número de componentes

Objetivo	Criterio
Visualización	Siempre 2 (o 3) componentes
Reducir sobreajuste	Validar con cross-validation
Compresión	Fijar un umbral de varianza explicada (90% o 95%)

```
pca = PCA(n_components=0.90) # retiene los componentes necesarios para el 90%
pca.fit(X)
print(f"Componentes seleccionadas: {pca.n_components_}")
```

Ventajas y desventajas de PCA

Ventajas	Desventajas
Simple, eficiente y bien fundamentado	Solo captura relaciones lineales
Elimina correlaciones entre features	Las componentes no siempre son interpretables
Reduce ruido descartando componentes de baja varianza	Requiere escalar los datos previamente
Permite visualizar datos de alta dimensión en 2D/3D	Puede perder información si la varianza no es el criterio correcto

5.3. Modelos Generativos

Un modelo generativo aprende la distribución de los datos para poder generar nuevas muestras, maximizando la verosimilitud:

$$L(\theta) = \log p_{\theta}(x) = \log \int p(x|z) p(z) dz$$

La idea central es aprender a transformar ruido gaussiano $p(z) = N(z | 0, I)$ en datos que sigan la distribución real.

PCA Probabilístico

El PCA probabilístico es el modelo generativo más sencillo: los datos x se generan desde una variable latente z de baja dimensión mediante una transformación lineal W más ruido gaussiano:

$$p(x | z) = N(x | Wz + \mu, \sigma^2 I)$$

$$p(z) = N(z | 0, I)$$

La distribución posterior del espacio latente:

$$p(z | x) = N(z | M^{-1}W^T(x - \mu), \sigma^{-2}M)$$

El entrenamiento se realiza por Maximum Likelihood:

$$\ln p(X | \mu, W, \sigma^2) = -(ND)/(2)\ln(2\pi) - (N)/(2)\ln|C| - (1)/(2)\sum_{n=1}^N(x_n - \mu)^T C^{-1}(x_n - \mu)$$

donde $C^{-1} = \sigma^{-2}I - \sigma^{-2}WM^{-1}W^T$.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca.fit(X)

Z = pca.transform(X)
X_rec = pca.inverse_transform(Z)
```

Modelos generativos modernos

El PCA probabilístico sienta las bases de los modelos generativos modernos, que reemplazan W por redes neuronales:

Variational Autoencoders (VAEs) — un encoder comprime los datos a un espacio latente z y un decoder los reconstruye. El espacio latente continuo y regularizado permite generar nuevas muestras interpolando en él.

Generative Adversarial Networks (GANs) — un generador $G_{(\theta_g)}$ transforma ruido en datos sintéticos y un discriminador $D_{(\theta_d)}$ distingue datos reales de generados. El generador mejora para engañar al discriminador.

Diffusion Models — aprenden a revertir un proceso de difusión que añade ruido gaussiano progresivamente. Son la base de modelos de imagen como Stable Diffusion.

Modelo	Mecanismo clave	Fortaleza
PCA probabilístico	Transformación lineal + ruido gaussiano	Simple, interpretable, solución cerrada
VAE	Encoder-decoder con espacio latente regularizado	Generación suave, interpolación en el latente
GAN	Juego generador vs discriminador	Alta calidad visual de las muestras generadas
Diffusion	Reversión iterativa de ruido	Estado del arte en generación de imágenes y audio